1. Hafta

C'den C++'a Geçiş

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- C ile C++ arasındaki derleme + bağlama modeli farkını ve C++'ın getirdiği tip güvenliği yaklaşımını açıklayabileceksiniz;
- `namespace` kullanarak isim çakışmalarını önleyebilecek, `std::cin`/`std::cout` akışlarıyla temel girdi/çıktı işlemleri yapabileceksiniz;
- Referansları (`int&`) C işaretçilerinden ayırt edip fonksiyon parametrelerinde doğru yerde kullanabileceksiniz;
- `auto` ile tip çıkarımını ve `const` ile değişmezlik sözünü uygun yerlerde uygulayabileceksiniz;
- Basit bir C++ bileşenini `.hpp`/`.cpp` olarak ayırıp `namespace` içinde CMake ile derlenebilir bir proje hâlinde kurabileceksiniz.

Bu dönemin ilk haftasındasınız. Daha önce hiç C++ görmediniz — ama sıfırdan başlamıyorsunuz: C'de bir kaynak dosyayı derleyip çalıştırmayı, işaretçi ve adres kavramını, `struct` ile veri toplamayı, fonksiyon tanımlayıp çağırmayı biliyorsunuz; Python'da bir yorumlayıcının kodunuzu satır satır çalıştırdığı bir deneyiminiz var; Algoritma Tasarımı dersinde bir problemi adımlara bölmeyi zaten öğrendiniz. Bu hafta bu bilgiyi C++'a *taşıyorsunuz* — yeni bir dili sıfırdan öğrenmiyor, aynı düşünme biçiminin C++'ta nerede aynı nerede farklı çalıştığını görüyorsunuz.

1.1 Neden C++? C/Python'dan Köprü

1 • Ön-öğretim — Neden Önemli?

Birden çok kaynak dosyadan oluşan bir proje derlerken, bir dosya tek başına sorunsuz derlendiği hâlde en sonda "undefined reference" hatası alırsınız — derlemenin ve bağlamanın iki ayrı aşama olduğunu bilmezseniz bu hata anlamsız görünür. Aynı biçimde C'nin gevşek tip kontrolünde uyumsuz bir atama ya da dönüşüm çalışma zamanına kadar sessiz kalıp orada patlar. Bu ünite "kod nasıl çalıştırılabilir hâle geliyor ve hatalar hangi aşamada yakalanıyor?" sorusunu netleştirir.

1 • Ön-öğretim — Ön Bilgi

Bu, dönemin ilk ünitesidir; dayanacağınız zemin doğrudan C ve Python deneyiminizdir. C'den `gcc` ile tek dosya derleme, `#include`, `printf`, `struct` ve işaretçi/adres kavramını hatırlayın; Python'dan ise değişkeni tip belirtmeden kullanmanın ve bir yorumlayıcının kodu satır satır çalıştırmasının nasıl bir deneyim olduğunu anımsayın — bu ikincisini, C++'ın derlenen/statik tipli yapısıyla karşıtlık kurmak için kullanacaksınız.

Bu döneme, elinizde zaten bir sermaye ile başlıyorsunuz: C'de bir kaynak dosyayı derleyip çalıştırmayı, işaretçi ve adres kavramını, `struct` ile veri toplamayı, fonksiyon tanımlayıp çağırmayı, `printf`/`scanf` ile ekrana yazıp klavyeden okumayı biliyorsunuz. Python'da da bir değişkeni tip belirtmeden kullanmanın, bir yorumlayıcının kodunuzu satır satır çalıştırmasının nasıl bir deneyim olduğunu gördünüz. Bu ders o sermayeyi sıfırdan bir dile taşımak değil, *aynı düşünme biçimini* C++'a taşımak üzerine kurulu. Bu ilk ünitede sorulacak soru şu: C zaten derlenip çalışan bir dilken, C++ hangi yeni sözü size veriyor da ayrı bir dil olarak öğrenmeye değer?

1.1.1 Aynı model, daha sıkı sözler

Önce ortak zemini netleştirin: C++ de tıpkı C gibi **derlenen** bir dildir — kaynak kodunuzu yazar, bir derleyiciden geçirir, bir çalıştırılabilir dosya elde edersiniz; Python'daki gibi kodunuzu satır satır yorumlayan bir program yoktur. Dolayısıyla C++'ı öğrenirken çalışma modelini baştan öğrenmiyorsunuz — `gcc` ile tek dosya derlemeye zaten alışkınsınız, aşağıda göreceğiniz akış bunun doğrudan genişlemesidir. Asıl fark çalışma modelinde değil, dilin size *verdiği sözlerde*: C++ derleyicisi, C'ninkine göre çok daha az şeyi "sizin sorumluluğunuza" bırakır, çok daha fazlasını kendisi denetler. Bu haftadan itibaren göreceğiniz her yeni özellik — sıkı tip kontrolü, referanslar, ileride sınıflar — aslında bu tek fikrin farklı görünümleridir: derleyiciye daha fazla söz verirseniz, derleyici sizin yerinize daha fazla hata yakalar.

Bu farkı soyut bırakmadan, aynı programın iki dildeki hâlini yan yana görün. Önce C sürümü:

```c
int main(void) {
    printf("Merhaba, C++!\n");
    return 0;
}
```

Şimdi aynı çıktıyı üreten C++ sürümüne bakın:

```cpp
int main() {
    std::cout << "Merhaba, C++!\n";
    return 0;
}
```

İki program da ekrana aynı satırı basar; ama araç farklıdır. C sürümünde çıktı işini `<stdio.h>` başlığından gelen `printf` üstlenir — ona bir *format dizgesi* (`"Merhaba, C++!\n"`) ve sıfır ek argüman verirsiniz; format dizgesindeki `%` işaretleri argümanlarla eşleşir (burada hiç yok, dizge sabit). C++ sürümünde ise çıktı işini `<iostream>` başlığından gelen `std::cout` akışı ve `<<` operatörü üstlenir — format dizgesi yoktur, yalnızca basılacak değer akışa "gönderilir". Bu ikinci yaklaşımı Ünite 2'de derinlemesine göreceksiniz; burada yalnızca "aynı sonuç, farklı araç" gözlemini yapmanız yeterli. `c_hali` ile `cpp_hali` etiketli iki kesit, sırasıyla `merhaba.c` ve `merhaba.cpp` dosyalarının tamamıdır — göreceğiniz gibi C++ sürümü tek satır daha kısa değil, ama kullandığı araç `printf`'ten köklü biçimde farklıdır.

1.1.2 Derleme + bağlama modeli

C'de tek dosyalık bir programı `gcc dosya.c -o program` gibi tek bir komutla derlediniz; bu komutun perde arkasında aslında iki ayrı aşama gizlenir, ve C++'ta — özellikle birden çok kaynak dosyanız olduğunda — bu iki aşamayı ayrı ayrı düşünmeniz gerekir.

Tanım: Derleme ve bağlama

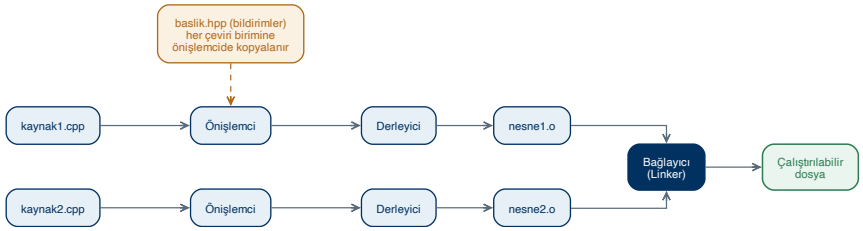
Derleme (*compilation*), her bir `.cpp` kaynak dosyasını ayrı ayrı makine diline çevirip bir *nesne dosyası* (*object file*, `.o`) üreten aşamadır. *Bağlama* (*linking*) ise bu nesne dosyalarını — ve varsa kullandıkları kütüphaneleri — birleştirip tek bir çalıştırılabilir dosya oluşturan sonraki aşamadır.

Bu iki aşamayı bir örnek üzerinden adım adım izleyin. Diyelim ki projenizde `a.cpp` ve `b.cpp` adlı iki kaynak dosyanız var. Derleyici önce `a.cpp`'yi tek başına, `b.cpp`'nin içeriğinden tamamen habersiz biçimde işler ve `a.o` adlı bir nesne dosyası üretir; ardından aynı şeyi `b.cpp` için de yapar ve `b.o`'yu üretir. Bu noktada elinizde iki ayrı, birbirinden kopuk makine kodu parçası vardır — henüz çalıştırılabilir bir dosya yoktur. Bağlayıcı (*linker*) devreye girer, `a.o` ile `b.o`'yu okur; eğer `a.cpp` içinde `b.cpp`'de tanımlı bir fonksiyonu çağırıyorsa, bağlayıcı bu çağrının nereye gideceğini — `b.o` içindeki ilgili makine kodu adresine — bağlar. Sonuçta tek bir çalıştırılabilir dosya ortaya çıkar. `a.cpp` değiştiğinde yalnızca `a.o` yeniden üretilir, `b.o` olduğu gibi kalır; bu, projeniz büyüdükçe derleme süresini kısaltan pratik bir kazançtır — bu haftanın 4. ünitesindeki `.hpp`/`.cpp` ayrımında bu kazancı çok daha somut biçimde göreceksiniz.

Peki `a.cpp`, `b.cpp`'de tanımlı bir fonksiyonu nasıl *bilir*? Burada devreye başlık dosyaları girer. `a.cpp`'nin başına bir `#include "b.hpp"` satırı koyarsanız, önişlemci (*preprocessor*) — derleyiciden önce çalışan, kaynak metni üzerinde salt metinsel değişiklik yapan araç — bu satırın yerine `b.hpp` dosyasının içeriğini olduğu gibi yapıştırır. `b.hpp` içinde fonksiyonun yalnızca *bildirimi* durur (adı, parametre tipleri, dönüş tipi) — gövdesi değil. Bu, derleyiciye "böyle bir fonksiyon bir yerde var, imzası budur, gövdesini bağlama aşamasında bulacaksın" demenin yoludur; derleyici bu söze güvenerek `a.cpp`'yi derler, gövdenin gerçekte nerede olduğuyla o an ilgilenmez — gövdeyi bulup çağrıyı gerçek adrese bağlamak bağlayıcının işidir. `a.cpp` kendi başlıklarıyla birlikte derleyiciye tek seferde sunulan hâline **çeviri birimi** denir — derleyici asıl olarak bu birimi görür, tek bir `.cpp` dosyasını değil.

çeviri birimi: bir `.cpp` dosyasının, içine dahil ettiği header'larla birlikte derleyiciye tek seferde sunulan hâli (*translation unit*).

Şekil 1.1 bu akışı uçtan uca özetliyor: birden çok `.cpp` kaynağı önişlemciden (header genişletme) geçip derleyiciye girer, her biri kendi `.o` dosyasını üretir, bağlayıcı bunları tek bir çalıştırılabilir dosyada birleştirir.



Şekil 1.1: Derleme + bağlama modeli: her `.cpp` ayrı derlenir, bağlayıcı nesne dosyalarını tek çalıştırılabilirde birleştirir.

Analoji

Bu modeli bir parça fabrikası ile montaj hattına benzetebilirsiniz. Her `.cpp` dosyası kendi fabrikasında, diğer fabrikalardan habersiz biçimde kendi parçasını üretir — bu, derleme aşamasıdır, çıktısı `.o` nesne dosyasıdır. Montaj hattı — bağlayıcı — bu ayrı ayrı üretilmiş parçaları alır, birbirine bağlar ve tek bir ürün, çalıştırılabilir dosyanız ortaya çıkar. Peki bir fabrika, hiç görmediği bir parçayla nasıl uyumlu üretim yapabiliyor? Cevap: elinde o parçanın *teknik şeması/krokisi* var — boyutları, bağlantı noktaları, hangi vidanın nereye gideceği. Header dosyası tam olarak bu krokidir: bir fabrika (`a.cpp`), diğer fabrikanın (`b.cpp`) iç üretim sürecini hiç görmeden, yalnızca krokiye (`b.hpp`'deki bildirimlere) bakarak kendi parçasını doğru biçimde üretebilir. Kroki yanlışsa (bildirim gerçek gövdeyle uyuşmuyorsa) montaj hattında — bağlama aşamasında — sorun patlak verir.

1.1.3 C++ zihniyeti: tip güvenliği ve ilk bakışta RAII

C'de `void*` işaretçileriyle ya da örtük tip dönüşümleriyle çalışırken derleyicinin oldukça toleranslı davrandığını fark etmiş olabilirsiniz: bir `int`'i bir `double` bekleyen bir yere vermek, ya da bir `void*`'ı hemen hemen her işaretçi tipine atamak genelde sessizce "çalışır". C++ derleyicisi burada daha sıkıdır: tip uyuşmazlıklarına, C'nin görmezden geldiği birçok durumda, çok daha hazır itiraz eder. Bu ilkeye **tip güvenliği** denir — ve bu haftadan başlayarak göreceğiniz `namespace`, `auto`, `const` gibi her yeni araç, aslında bu sıkılığın farklı bir görünümüdür.

tip güvenliği: bir programlama dilinin, tip uyumsuzluklarını çalışma zamanı yerine mümkün olduğunca derleme zamanında yakalama derecesi (*type safety*); C++ bu konuda C'den daha katıdır.

Bu haftadan itibaren adını sık duyacağınız bir kavrama da burada, yalnız bir cümlelik bir tanıtımla değinin — konunun kendisine değil, yalnızca adına ve tek cümlelik fikrine şimdilik aşina olmanız yeterlidir: **RAII** Bu haftaki kodunuzda RAII'nin somut bir örneğini görmeyeceksiniz — yalnızca ismini ve tek cümlelik fikrini tanıyın; mekanizmanın kendisini Hafta 4'te ayrıntılı işleyeceksiniz. Otel oda kartını düşünün: kartı aldığınızda oda size aittir, oteli terk edip kartı geri verdiğinizde oda otomatik boşa çıkar — kaynağın alınışı ile bırakılışı, bir nesnenin (burada: kartın) ömrüne bağlanmıştır. C++'ın sınıfları (Hafta 3'ten itibaren) bu fikri tam olarak bu şekilde kullanacak.

RAII: bir kaynağın (bellek, dosya gibi) ömrünü bir nesnenin ömrüne bağlayan C++ tasarım ilkesi (*Resource Acquisition Is Initialization*) — nesne oluşturulduğunda kaynak alınır, nesne yok edildiğinde kaynak otomatik bırakılır.

1.1.4 Python'a kıyasla: derleme zamanında yakalanan hatalar

Python'da bir değişkeni tip belirtmeden tanımlarsınız ve yorumlayıcı programı satır satır çalıştırır; bir tip hatası — örneğin bir sayıyı bir metinle toplamaya çalışmak — ancak o satır *gerçekten çalıştığında* ortaya çıkar. C++'ta ise her değişkenin bir tipi

vardır ve derleyici, programınızı hiç çalıştırmadan, yalnızca kaynak kodu okuyarak böyle bir uyumsuzluğu tespit edebilir. Bu fark küçük bir ayrıntı değil, günlük iş akışınızı doğrudan etkiler: Python'da bir hata, o kod yolunu tetikleyen belirli bir girdi verilene kadar gizli kalabilir; C++'ta aynı sınıftan birçok hata, programı hiç çalıştırmadan, derleme anında — yani mümkün olan en erken anda — yüzeye çıkar. Bunun bedeli de vardır: Python'da hızlıca yazıp denediğiniz bir satırı C++'ta önce derleyiciyi tatmin edecek biçimde yazmanız gerekir; ama karşılığında, üretime gitmeden önce yakalanan bir hata sınıfı kazanırsınız.

1.1.5 CMake'e ilk bakış

Derleme sistemi: Derleme+bağlama adımlarını proje genelinde otomatikleştiren araç (*build system*); bu ders boyunca kullanacağınız araç CMake'tir.

Birden çok `.cpp` dosyanız olduğunda, her birini elle `gcc/clang++` komutuyla derleyip doğru sırayla bağlamak hızla sıkıcı ve hataya açık hâle gelir. **Derleme sistemi** tam olarak bunu otomatikleştirir: hangi kaynak dosyaların, hangi standartla, hangi sırayla derlenip bağlanacağını bir yapılandırma dosyasında (`CMakeLists.txt`) tanımlarsınız, komutu elle yazmak yerine bu dosyayı çalıştırırsınız. Bu haftalık ünitede CMake'i yalnızca kavramsal düzeyde tanıyorsunuz — gerçek bir `CMakeLists.txt` dosyasını Ünite 4'te, kendi kütüphane iskeletinizi kurarken elinizle yazacaksınız.

Ortak Hata / Tuzak

C'den gelen bir alışkanlıkla `printf/scanf` tarzı format dizgeli I/O'yu C++'ta da birebir sürdürmeye çalışmak yaygın bir tuzaktır. Derlenir, çalışır — C++ programınızda `<stdio.h>` kullanmanız teknik olarak yasak değildir — ama bu haftadan itibaren göreceğiniz `std::cin/std::cout` akışlarının kazandırdığı tip güvenliğinden (Ünite 2) vazgeçmiş olursunuz: `printf` bir format dizgesindeki `%d` ile gerçek argüman arasındaki uyumu *derleme zamanında değil*, doğrudan kontrol etmez; `std::cout <<` ise argümanın gerçek tipine bakarak doğru biçimde basar, format dizgesi hiç yoktur. Bu ders boyunca C++ kodunuzda `<iostream>` akışlarını tercih edin.

Bu ünitede gördüğünüz fikir kısaca şudur: C++ çalışma modeli olarak C'den ayrılmaz — ikisi de derlenip bağlanır — ama derleyicinin size verdiği sözler çok daha sıkıdır. Derleme + bağlama modelini ve header dosyalarının bu modeldeki rolünü (yalnız bildirim taşımak, önişlemcide kopyalanmak) bu ünitede gördünüz; bu, ilerleyen üç ünitede göreceğiniz her yeni araç için ortak zemindir. Gelecek ünitede bu modelin üstüne `namespace` ve `<iostream>` ile ilk gerçek C++ kodunuzu yazacaksınız.

3 • Katılım — ⌨ Şimdi Siz Deneyin

Bir projede `a.cpp` ve `b.cpp` var; `a.cpp`, `b.cpp` içinde tanımlı bir fonksiyonu çağırıyor ve bu fonksiyonun bildirimini bir başlık dosyasından alıyor. (a) Derleyici `a.cpp`'yi tek başına derlerken fonksiyonun gövdesini görmediği hâlde neden hata vermez? (b) `b.cpp`'yi bağlama adımına vermeyi unutursanız "undefined reference" hatası hangi aşamada çıkar?

Geribildirim. (a) Başlıktaki bildirim derleyiciye fonksiyonun imzasını — adını, parametrelerini, dönüş tipini — verir; bu, `a.cpp`'yi bir çeviri birimi olarak derleyip `a.o` üretmeye yeter, gövde bu aşamada gerekmez. (b) Gövde *bağla-*

ma (linker) aşamasında aranır; `b.o` bağlayıcıya verilmezse fonksiyonun tanımı bulunamaz ve bağlayıcı "undefined reference" der. Yani derleme başarılı, bağlama başarısızdır — hatanın iki aşamadan hangisine ait olduğunu ayırt etmek işin anahtarıdır.

Kendinizi Sınayın 1.1

C++'ta derleme neden tek adım değil de derleme + bağlama olarak iki ayrı aşamadır? Header dosyaları bu modelde ne işe yarar? Yanıtınızda çeviri birimi ve nesne dosyası kavramlarını kullanın.

5 • İzleme — Aktarım — Başka Nerede?

Bu derleme + bağlama modeli tek bir programa özgü değildir; birden çok dosyadan oluşan her C/C++ projesinde aynı biçimde işler. Ünite 4'te kendi `.hpp`/`.cpp` iskeletinizi kurarken bu modeli somut olarak yaşayacak, aynı "bildirim başlıkta, gövde kaynakta" ayrımını elinizle uygulayacaksınız. Hafta 3'te `Kitap` sınıfı da tam bu çeviri birimi mantığıyla — bildirimi `.hpp`'de, tanımı `.cpp`'de — ayrılarak derlenecek; bugün gördüğünüz aşama ayrımı, dönem boyunca yazacağınız her çok dosyalı projenin temelidir.

1.2 `namespace`, `<iostream>`, `std::string`, Referanslar

1 • Ön-öğretim — Neden Önemli?

İki farklı kütüphaneden gelen aynı isimli iki tanım — örneğin iki ayrı `topla` ya da iki `Nokta` — aynı projede karşılaşınca C'de tek çareniz isimleri elle uzatmaktır (`kutuphane1_topla` gibi); hantal ve hataya açık bir yoldur. Üstelik `scanf("%d", &x)` yerine yanlışlıkla `scanf("%f", &x)` yazmak sessizce çöp değer ya da çökme üretir; derleyici sizi uyarmaz. Bu ünite isim çakışmasını `namespace` ile, tip uyuşmazlığı riskini de tip-güvenli `std::cin`/`std::cout` akışlarıyla çözer.

1 • Ön-öğretim — Ön Bilgi

Bu üniteye girmeden önce Ünite 1'in derleme zamanında hata yakalama (tip güvenliği) fikrini taze tutun — bu ünitedeki akışlar onun somut bir uygulamasıdır. C'den `printf`/`scanf` ve format dizgesi, `char*`/karakter dizisi, fonksiyona değer ve işaretçiyle parametre geçişini hatırlayın. Python'dan ise modül/isim alanı sezgisini (`import` ile isimlerin bir alana ait olması) anımsayın; bu, birazdan göreceğiniz `namespace` fikrine karşılık gelir.

Önceki ünitede C++'ın derleme + bağlama modelini ve genel zihniyetini gördünüz; şimdi ilk gerçek C++ araçlarını elinize alma zamanı. Bu ünitede dört yeni kavramla tanışacaksınız: isim çakışmasını çözen `namespace`, klavye/ekran ile konuşmanızı sağlayan `<iostream>`, C'nin ham karakter dizisine kıyasla çok daha rahat bir metin tipi olan `std::string`, ve C işaretçilerinden farklı, ama benzer bir işi gören *referanslar*.

1.2.1 namespace: isim çakışmasını çözmek

C'de tüm fonksiyon ve değişken adları — birkaç istisna dışında — tek, küresel bir isim havuzunda yaşar: iki farklı dosyada aynı isimde iki fonksiyon tanımlarsanız, bunları aynı programda birleştirdiğinizde bağlayıcı bir çakışma hatası verir. C++ bu soruna bir çözüm ekler:

Tanım: Ad alanı

Bir *ad alanı* (*namespace*), aynı isimli tanımların farklı "bölgelerde" çakışmadan var olmasını sağlayan bir isimlendirme kapsayıcısıdır. Bir ad alanı içindeki bir isme dışarıdan erişmek için `adAlani::isim` biçiminde — iki iki nokta üst üste (*scope resolution operator*) ile — tam nitelenmiş adı kullanırsınız.

C++ standart kütüphanesinin tamamı — `cout`, `cin`, `string` ve daha fazlası — `std` adlı bir ad alanının içinde tanımlıdır; bu yüzden bu haftaki kodlarda sürekli `std::cout`, `std::string` gibi `std::` önekiyle karşılaşacaksınız. Bunun nedenini somut bir senaryoyla görün: diyelim ki kendi kodunuzda, tamamen farklı bir amaçla, `count` adlı bir fonksiyon ya da değişken tanımladınız. Standart kütüphanenin de kendi `std::count` adlı bir fonksiyonu vardır (bir dizideki elemanları sayan). Bu iki `count` aynı ad alanında olsaydı derleyici hangisini kastettiğinizi karıştırırdı; ama biri `std` ad alanında, diğeri global kapsamda olduğu için — `std::count` ile sizin `count`'unuz — derleyici ikisini asla karıştırmaz.

Analoji

Ad alanını bir soyadına ya da şehir koduna benzetebilirsiniz. Aynı isimde birçok kişi olabilir — birden fazla "Ali" — ama soyadı ya da hangi şehirden geldiği hangi kişiden bahsedildiğini netleştirir; "Ali Yılmaz, İstanbul" ile "Ali Kaya, Trabzon" hiçbir zaman karışmaz, çünkü tam ad her ikisini de ayırt eder. `std::cout` da tam olarak böyledir: `cout`, `std` ad alanına "kayıtlıdır"; siz kendi kodunuzda farklı bir ad alanında (ya da hiç ad alanı olmadan) aynı kısa adı kullansanız bile iki tanım asla çakışmaz.

Kodunuzun başına `using namespace std;` yazıp `std::` önekini her seferinde tekrarlamaktan kurtulabileceğinizi bir yerlerde görmüş olabilirsiniz; bu satırı özellikle *header dosyalarında* yazmaktan kaçının. Sebebini somutlaştırın: bir header dosyasına `using namespace std;` yazarsanız, bu satır o header'ı include eden *her* `.cpp` dosyasına — önişlemci aracılığıyla — kopyalanır; sonuçta o dosyaların hepsinde `std` ad alanındaki *tüm* isimler önek gerekmeden erişilebilir hâle gelir, ve bu, tam olarak ad alanlarının önlemeye çalıştığı isim çakışması riskini geri getirir — üstelik bu kez riski o header'ı kullanan herkese, sizin haberiniz bile olmadan yaymış olursunuz. Bunun yerine iki güvenli yol tercih edin: `std::` önekini her seferinde yazmak (en açık, en güvenli yol), ya da yalnızca sık kullandığınız birkaç ismi seçici biçimde içeri almak — `using std::cout;` gibi — ve bunu da yalnızca `.cpp` dosyalarında, header'larda değil. Bu haftaki kod örnekleri `std::` önekini açıkça yazan yaklaşımı izler.

1.2.2 <iostream>: std::cin/std::cout ile giriş/çıkış

C'de klavyeden okuma ve ekrana yazma işini `<stdio.h>` başlığından gelen `scanf`/`printf` üstlenirdi; C++'ta bu iş `<iostream>` başlığından gelen *akış* nesneleriyle yapılır.

Tanım: Akış

Bir *akış* (*stream*), veriyi bir kaynaktan (ör. klavye) hedefe (ör. ekran) sırayla taşıyan bir soyutlamadır. `std::cin` standart girdi akışını (klavye), `std::cout` standart çıktı akışını (ekran) temsil eder; `<<` operatörü veriyi akışa doğru *gönderirken*, `>>` operatörü veriyi akıştan bir değişkene *çeker*.

Analoji

Akış yönünü bir postane gönderim/teslim kutusuna benzetebilirsiniz. `<<` operatörünün oku sağa, hedefe doğru bakar — tıpkı bir zarfı postane kutusuna *gönderdiğinizde* zarfın kutuya doğru hareket etmesi gibi: `std::cout << deger;` değeri ekrana doğru gönderir. `>>` operatörünün oku ise sola, kaynaktan değişkene doğru bakar — tıpkı postaneden bir zarfı *teslim aldığınızda* zarfın size doğru gelmesi gibi: `std::cin >> degisken;` klavyeden okunan değeri değişkene doğru "teslim alır". Okun yönünü hatırlarsanız, `<<` ile `>>`'yi hiçbir zaman karıştırmazsınız.

Bu iki akışı gerçek bir örnekte görün:

```
19  std::string isim;
20  std::cout << "Adiniz nedir? ";
21  std::cin >> isim;
22  std::cout << "Merhaba, " << isim << "!\n";
```

Bu üç satırı adım adım izleyin. İlk satır bir `std::string` değişkeni — `isim` — boş bir dizge olarak tanımlar (bir sonraki bölümde `std::string`'e döneceksiniz). İkinci satır `std::cout << "Adiniz nedir? ";` ekrana bir istem yazdırır ve *yeni satıra geçmez* — kullanıcı yanıtını aynı satırda yazabilsin diye. Üçüncü satır `std::cin >> isim;` klavyeden okunan metni `isim` değişkenine yazar; `std::cin` boşluk karakterine kadar okur, yani kullanıcı "Ada Lovelace" yazsa bile `isim` yalnızca "Ada"yı alır (boşluktan sonrası bir sonraki `>>` çağrısını bekler). Son satır okunan ismi bir selamlama içinde geri basar.

C'nin `scanf`'ıyla kıyaslayın: `scanf("%d", &sayi);` yazarken hem doğru format belirtecini (`%d`) elle seçmeniz hem de değişkenin adresini (`&sayi`) elle vermeniz gerekir — ikisinden biri yanlışsa (`%d` yerine `%s` yazmak gibi) derleyici size çoğu zaman *hiç* itiraz etmez, hata yalnızca çalışma zamanında, genellikle programın çökmesi ya da anlamsız bir değer okuması biçiminde ortaya çıkar. `std::cin >> sayi;` yazarken ise ne bir format belirteci ne bir `&` yazarsınız — akış operatörü `sayi`'nin gerçek tipine (derleme anında bilinen) bakarak doğru okuma biçimini kendisi seçer. Bu, Ünite 1'de gördüğünüz tip güvenliği ilkesinin somut bir örneğidir: C++ akışlarında format-tip uyumsuzluğu riski derleyici tarafından *kökten* ortadan kaldırılmıştır, çünkü ortada eşleştirilmesi gereken bir format dizgesi hiç yoktur.

1.2.3 `std::string`: bir tadımlık

C'de metin bir `char` dizisi (`char isim[50];`) ya da bir `char*` işaretçisiyle temsil edilir; uzunluğunu kendiniz takip etmeniz, birleştirmek için `strcat` gibi fonksiyonları elle çağırmanız, arabellek taşmalarına (*buffer overflow*) karşı dikkatli olmanız gerekir. C++'ın `std::string` tipi bu işi büyük ölçüde sizin için üstlenir — kendi belleğini kendisi yönetir, boyutu gerektikçe kendisi büyür.

```
26  std::string ad = "Ada";
27  std::string soyad = "Lovelace";
```

```
28    std::string tamAd = ad + " " + soyad;
29    std::cout << tamAd << " (" << tamAd.size() << " karakter)\n";
```

Bu üç satırı izleyin: `ad + " " + soyad` ifadesi, C'deki gibi `strcat(strcat(buf, " "), soyad)` yazmanıza gerek kalmadan, üç dizgeyi tek bir `+` zinciriyle birleştirir — sonuç yeni bir `std::string` nesnesidir, kendi belleğini kendisi ayırmıştır. `tamAd.size()` ise dizgenin kaç karakter uzunluğunda olduğunu doğrudan bir üye fonksiyon çağrısıyla söyler — C'de bunun karşılığı `strlen(buf)` çağırmak, ya da diziyi elinizle gezip sonlandırıcı sıfır karakterini aramak olurdu. Bu haftalık amaç `std::string`'in tüm arayüzünü öğrenmek değil, yalnızca C'nin ham karakter dizisine kıyasla ne kadar daha okunabilir olduğunu görmenizdir; sınıfın kendisini — kendi tipinizi nasıl tasarlayacağınızı — Hafta 3'te ayrıntılı işleyeceksiniz.

1.2.4 Referanslar: C işaretçilerinden farklı bir bağlama

C'de bir fonksiyonun bir değişkeni *değiştirebilmesi* için o değişkenin adresini işaretçiyle geçirdiniz: `void ikiyeKatla(int* p) { *p = *p * 2; }` ve çağırırken `ikiyeKatla(&sayi);` yazdınız. C++ aynı işi yapmanın ikinci, daha az sözdizimsel gürültülü bir yolunu sunar:

Tanım: Referans

Bir *referans* (*reference*, `T&`), var olan bir değişkene verilen ikinci bir isimdir (takma ad) — bir işaretçi gibi ayrı bir "adres değeri" taşımaz, doğrudan o değişkenin kendisidir. Bir referans tanımlandığı anda bir değişkene bağlanmak *zorundadır*, boş (*null*) olamaz, ve bir kere bağlandıktan sonra başka bir değişkeni göstermek üzere *yeniden bağlanamaz*.

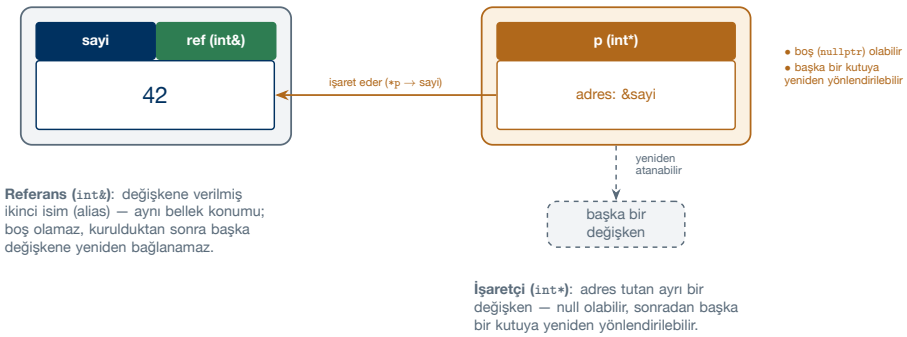
Analoji

Referans ile işaretçi arasındaki farkı bir lakap ile yazılı bir ev adresine benzetebilirsiniz. Bir kişinin lakabı her zaman o kişinin kendisidir — lakap boş olamaz (bir lakap ille birine ait olmalıdır) ve bir kere birine takıldıktan sonra "yeniden takılamaz" (aynı lakap başka birine sözlü olarak devredilemez, yeni bir lakap gerekir). Buna karşılık bir kağıda yazılmış ev adresi (işaretçi) *boş* bırakılabilir — kağıt elinizde ama üstünde hiçbir adres yazmıyor olabilir, hiçbir yeri göstermez — ve bu kağıda yeni bir adres yazıp *başka bir eve yeniden yönlendirebilirsiniz*. Adresi okuyup gerçek eve gitmek için ekstra bir adım gerekir (dereference, C'de `*p`); lakabı kullanmak içinse ekstra bir adım gerekmez — lakabın kendisi zaten kişinin ta kendisidir.

Şekil 1.2 bu iki bağlama biçimini yan yana gösteriyor: solda bir değişkene ikinci bir isim olarak iliştirilmiş referans, sağda ayrı bir adres tutan, boş olabilen ve yeniden yönlendirilebilen bir işaretçi.

Bu farkın sözdizimine nasıl yansıdığını bir fonksiyon parametresi üzerinden görün:

```
6  void ikiyeKatla(int& sayi) {
7     sayi = sayi * 2;
8  }
9
10 void referansOrnegiCalistir() {
11    int sayi = 21;
12    ikiyeKatla(sayi);
```



Şekil 1.2: Referans (takma ad, boş olamaz, yeniden bağlanamaz) ile işaretçi (adres, boş olabilir, yeniden yönlendirilebilir) arasındaki fark.

```
13     std::cout << "Ikiye katlanan sayi: " << sayi << '\n';
14 }
```

`ikiyeKatla(int& sayi)` imzasındaki `&` işareti, `sayi` parametresinin bir `int` *referansı* olduğunu söyler — yani `sayi`, çağıran taraftaki gerçek değişkenin ikinci bir ismidir, kendi başına ayrı bir kopya değildir. Gövdedeki `sayi = sayi * 2;` satırı, tıpkı sıradan bir yerel değişkene atama yaparmış gibi görünür — ama `sayi` bir referans olduğu için bu atama gerçekte çağıran taraftaki değişkeni değiştirir. Şimdi çağıran tarafa bakın: `ikiyeKatla(sayi);` satırında ne `&sayi` (adres alma) ne `*` (dereference) yazarsınız — referans parametreli bir fonksiyonu çağırmak, sıradan bir değer geçirmekle birebir aynı görünür; farkı yaratan tamamen fonksiyonun *imzasındaki* `&`'dır. Bu, C'nin işaretçi tabanlı `ikiyeKatla(&sayi)` çağrısına kıyasla hem daha az gürültülü hem de daha güvenlidir: bir referans boş olamayacağı için, fonksiyon gövdesinde "acaba `sayi` geçerli bir yeri mi gösteriyor" diye endişelenmenize gerek yoktur — C'deki `if (p != NULL)` kontrolüne referanslarda hiç ihtiyaç duymazsınız, çünkü öyle bir durumun (boş referans) dilde bir karşılığı yoktur.

Ortak Hata / Tuzak

Bir referansı, C'deki bir işaretçi gibi "yeniden atayarak" başka bir değişkeni gösterecek şekilde değiştirmeye çalışmak yaygın bir kavram karışıklığıdır. `int& r = a; r = b;` yazdığınızda, bu satır `r`'yi `b`'yi gösterecek şekilde *yeniden bağlamaz* — `r` zaten `a`'nın kendisi olduğu için, `r = b;` aslında `a`'nın değerinin üzerine `b`'nin değerini yazar (`a`, `b` ile aynı değere sahip olur). Bir işaretçide `p = &b;` yazmak `p`'nin *gösterdiği yeri* değiştirirken, bir referansta böyle bir "yeniden gösterme" operasyonu yoktur — referans kurulduğu andan itibaren hep aynı değişkenin ikinci ismi olarak kalır.

Bu ünitede gördüğünüz fikir kısaca şudur: `namespace` isim çakışmasını çözer, `<iostream>` akışları tip güvenli bir giriş/çıkış sağlar, `std::string` C'nin ham karakter dizisine kıyasla çok daha güvenli bir metin tipidir, referanslar ise C işaretçilerinin yaptığı işin bir bölümünü — bir değişkeni fonksiyona değiştirilebilir biçimde geçirmeyi — boş olamayan, yeniden bağlanamayan, daha az gürültülü bir sözdizimiyle yapar. Gelecek ünitede `auto` ile derleyicinin tip çıkarımına ve `const` ile değişmezlik sözüne bakacaksınız.

3 • Katılım **Şimdi Siz Deneyin**

(a) `scanf("%d", &x)` yerine yanlışlıkla `%f` yazarsanız ne olur; aynı hatayı `std::cin >> x;` ile yapabilir misiniz? (b) `void ikiyeKatla(int& n)` fonksiyonunu `main` içinde `int sayi = 21;` için çağırırken `ikiyeKatla(&sayi)` mi yoksa `ikiyeKatla(sayi)` mi yazarsınız, neden?

Geribildirim. (a) `%f`/`%d` uyuşmazlığını derleyici yakalamaz; çalışma zamanında tanımsız davranış (çöp değer ya da çökme) doğar. `std::cin >> x;` yazarken ise akış hedef değişkenin tipini kendisi bildiği için ortada bir format dizgesi hiç yoktur — bu hatayı yapmanız mümkün değildir. (b) `ikiyeKatla(sayi);` yazarsınız: referans parametresine çağıran tarafta `&` eklenmez, çünkü referans zaten `sayi`'nın ikinci bir ismidir (takma ad), bir işaretçi değildir. Çağrıdan sonra `sayi` 42 olur.

Kendinizi Sınayın 1.2

`std::cin >> sayi;` ile C'nin `scanf("%d", &sayi);`'ı arasındaki temel fark nedir? Yanıtınızda tip güvenliği kavramını kullanın.

5 • İzleme **Aktarım — Başka Nerede?**

`namespace`, dönem boyunca kendi kodunuzu düzenlemenin temel aracıdır: Ünite 4'te `Nokta`'yı bir `geometri` ad alanına, Hafta 3'te `Kitap`'ı bir `kitaplik` ad alanına koyacaksınız. Referanslar da nesneleri kopyalamadan fonksiyonlara geçirmenin standart yoludur — ileride Kitaplık projesinde büyük `Kitap` nesnelerini `const` referansla geçirerek hem kopya maliyetinden kaçınacak hem değiştirilmelerini önleyeceksiniz. Bu, bir sonraki ünitenin `const` referans konusuna doğrudan köprüdür.

1.3 `auto` ve Tip Çıkarımı; `const`; Modern C++ Tadımlığı

1 • Ön-öğretim **Neden Önemli?**

Bir fonksiyona büyük bir `std::string`'i değere göre (`std::string mesaj`) geçirmek her çağrıda tam bir kopya üretir — büyük metinlerde bu maliyet birikir; üstüne, fonksiyonun bu metni yanlışlıkla değiştirmeyeceğini nasıl garanti edersiniz? Ayrıca ileride karşılaşacağınız uzun tip adlarını (yineleyici tipleri gibi) elle yazmak yorucu ve hataya açıktır. Bu ünite `const` referansla iki sorunu birden — kopyasızlık ve değişmezlik — `auto` ile de tip adını derleyiciye çıkartma sorununu çözer.

1 • Ön-öğretim **Ön Bilgi**

Bu ünite doğrudan Ünite 2'nin üstüne kurulur: referansları (`const` referans onların üstüne oturur) ve `std::string`/`std::cout`'u taze tutun. C'den değişken tanımında tipi zorunlu belirtmeyi (`auto` tam bu yükü kaldırır) ve `const` anahtar sözcüğünü hatırlayın. Python'dan ise dinamik tipleme deneyimini anımsayın

— birazdan `auto`'nun dinamik tip *olmadığını*, derleme zamanında sabitlenen tek bir tip çıkardığını bu karşıtlıkla ayırt edeceksiniz.

Önceki ünitede referanslarla değişkenlere ikinci bir isim vermeyi öğrendiniz; bu ünitede iki araçla daha tanışacaksınız: derleyicinin tipi sizin yerinize çıkardığı `auto`, ve bir değişkenin değişmeyeceğine dair derleyiciye verdiğiniz bir söz olan `const`. Ünitenin sonunda, C'nin klasik indeksli döngüsüne kıyasla çok daha sade bir gezinme biçimi olan aralık tabanlı `for`'a da kısaca bakacaksınız.

1.3.1 `auto`: derleyici tipi sizin yerinize çıkarır

C'de her değişkeni tanımlarken tipini elle yazarsınız: `int toplam = a + b;`. Bu çoğu zaman sorun değildir — `int` kısa bir tip adıdır — ama tip adı uzadıkça (ileride göreceğiniz yineleyici tipleri gibi) bu tekrarı elle yazmak hem yorucu hem de hataya açık hâle gelir: tip adını yanlış yazarsanız ya da ifadenin gerçek tipiyle uyuşmayan bir tip yazarsanız, ya derleme hatası alırsınız ya da — daha kötüsü — örtük bir dönüşümle sessizce yanlış bir tipte bir değişken elde edersiniz. C++11'den itibaren, bu tekrarı ortadan kaldıran bir araç var:

Tanım: Tip çıkarımı

Tip çıkarımı (*type inference*), derleyicinin bir ifadenin sağ tarafından (ilk değerinden) değişkenin tipini kendisinin belirlemesidir. `auto` anahtar kelimesi bir değişken tanımında tip adının yerine yazılır; derleyici o değişkene, `=` işaretinin sağındaki ifadenin tipini atar.

Analoji

`auto`'yu bir otomatik vitesli araca benzetebilirsiniz. Sürücü — yani siz, programcı — hâlâ nereye gideceğinize, hangi yoldan gideceğinize karar verirsiniz; `auto`, yalnızca vites seçimini — burada: değişkenin tipini — sizin yerinize, o anki yol koşuluna (ifadenin sağ tarafına) bakarak yapar. Vites otomatik seçilse de araç hâlâ sizin yönlendirdiğiniz yere gider; tip otomatik çıkarılsa da program hâlâ sizin yazdığınız mantıkla çalışır — `auto` yalnızca tip adını elle yazma işini üstlenir, mantığı değiştirmez.

Bu fikri küçük bir örnekte görün:

```
13  auto a = 7;
14  auto b = 35;
15  auto toplam = a + b;
16  std::cout << "Toplam: " << toplam << '\n';
```

Bu üç satırı adım adım izleyin. `auto a = 7;` satırında `7` bir tamsayı sabiti olduğu için derleyici `a`'ya `int` tipini verir — tıpkı `int a = 7;` yazmış gibi, yalnızca tip adını siz değil derleyici yazmıştır. `auto b = 35;` için de aynı akıl yürütme geçerlidir, `b` de `int` olur. Üçüncü satırda `auto toplam = a + b;` yazdığınızda derleyici önce `a + b` ifadesinin tipini hesaplar — iki `int`'in toplamı yine bir `int`'tir — sonra `toplam`'a bu tipi verir. Önemli bir noktayı burada vurgulayın: `auto` tipi *çalışma zamanında değil*, tıpkı elle yazılmış bir tip gibi *derleme zamanında* belirlenir; `toplam`'ın çalışma zamanında "hangi tipte olduğuna karar verdiği" bir an yoktur — bu, `auto`'yu Python'daki dinamik tiplemeden ayıran temel farktır. Python'da bir değişken çalışma zamanında farklı tiplerde değerler tutabilir; C++'ta `auto` ile tanımlanan bir değişkenin tipi

bir kez belirlenir ve o değişkenin ömrü boyunca *sabit kalır* — yalnızca tip adını kim yazdığı (siz mi, derleyici mi) değişir.

`auto`'nun ne zaman kullanışlı olduğunu şimdiden hissetmiş olabilirsiniz: tip adı uzun ya da karmaşık olduğunda (ileride yineleyici tiplerinde bunu çok daha belirgin göreceksiniz) elle yazmak yerine `auto`'ya bırakmak kodu kısaltır ve tip adını yanlış yazma riskini ortadan kaldırır. Ama dikkatli olmanız gereken bir yan etkisi de vardır: `auto` kullanılan bir satırı okuyan biri, değişkenin gerçek tipini görmek için sağ taraftaki ifadeyi (bazen başka bir fonksiyonun dönüş tipini) takip etmek zorunda kalır — tip artık satırda *açıkça yazılı değildir*, gizlenmiştir. Kısa, apaçık ifadelerde (`auto toplam = a + b;` gibi) bu bir sorun değildir; ama karmaşık, iç içe bir ifadenin sonucuna `auto` verdiğinizde, okunabilirlik lehine tip adını elle yazmayı tercih edebilirsiniz.

1.3.2 `const`: değişmezlik sözü

Tanım: Sabitlik

`const` anahtar kelimesi, bir değişkenin ya da parametrenin değerinin değiştirilmeyeceğine dair derleyiciye verilen bir garantidir (*const-correctness*). `const` olarak işaretlenmiş bir değişkene atama yapmayı deneyen bir satır *derlenmez* — hata çalışma zamanında değil, derleme anında yakalanır.

Analoji

`const`'u duvara çakılı bir tabelaya benzetebilirsiniz. Bir tabela bir kez yazılıp asıldıktan sonra üzerinde oynama yapılmaz — kimse gelip tabelanın üstüne farklı bir yazı eklemez, tabela ne yazıyorsa o kalır. `const` bir değişken ya da parametre de tam olarak böyle bir mühürle işaretlenmiştir: "bu değeri değiştirmeyeceğime söz veriyorum" — ve bu sözü siz değil, derleyici sizin yerinize, kodun her satırında zorla uygular.

Bu fikrin en pratik kullanımı, bir fonksiyona bir referans parametre geçerken ortaya çıkar. Önceki ünitede `int&` gibi sade bir referans parametre gördünüz; şimdi `const` ile birleşmiş hâline bakın:

```
6 void mesajYazdir(const std::string& mesaj) {
7     std::cout << mesaj << '\n';
8 }
```

Bu tek satırlık fonksiyon imzasını parça parça okuyun. `mesajYazdir` bir `std::string` parametresi alıyor — eğer bu parametreyi `std::string mesaj` (referans olmadan, *değere göre*) olarak yazsaydınız, fonksiyon çağrıldığı her seferde dizgenin *tam bir kopyası* oluşturulurdu; dizge uzunsa bu kopyalama gereksiz bir bellek ve zaman maliyetidir. `&` eklediğinizde — `std::string& mesaj` — artık kopya alınmaz, `mesaj` çağırandaki gerçek dizgenin ikinci bir ismi olur; ama bu kez fonksiyon gövdesi isterse o dizgeyi *değiştirebilir* — çağırana ait veriye yazma izniniz olur, ki bu haldeki bir yazdırma fonksiyonu için hiç istenmeyen bir yan etkidir. `const std::string& mesaj` yazdığınızda ise iki faydayı *aynı anda* elde edersiniz: kopya alınmaz (referans olduğu için) *ve* fonksiyon gövdesi `mesaj`'ı değiştiremez (derleyici bunu `const` sayesinde zorla uygular) — `mesajYazdir` gövdesinde `mesaj += "!";` gibi bir satır yazmaya kalksanız, program derlenmez.

Ortak Hata / Tuzak

Bir fonksiyon parametresinin yalnızca okunacağını, hiç değiştirilmeyeceğini bildiğiniz hâlde onu düz `std::string mesaj` (değere göre) ya da `std::string& mesaj` (const'suz referans) olarak bırakmak yaygın bir verimsizliktir/risktir. Birincisinde her çağrıda gereksiz bir kopya alınır; ikincisinde ise fonksiyonun çağırandaki veriyi *değiştirmeyeceğine* dair kodda hiçbir garanti yoktur — gövdeyi okumadan emin olamazsınız. Bir parametreyi yalnızca okuyacaksanız ve tipi küçük değilse (`std::string`, ileride göreceğiniz kullanıcı tanımlı tipler gibi), `const T&` kalıbı — kopyasız *ve* değişmez — neredeyse her zaman doğru tercihtir.

1.3.3 Küçük bir modern C++ tadımlığı

C'de bir diziyi baştan sona gezmek için klasik indeksli döngüyü yazarsınız: `for (int i = 0; i < n; i++) { ... dizi[i] ... }`. Bu döngüde üç ayrı şeyi elle yönetirsiniz: başlangıç indeksi, bitiş koşulu, artış adımı — ve her biri yanlış yazılabilir (bir-kayması hatası, *off-by-one*, bunun en yaygın örneğidir). C++11'den itibaren, yalnızca elemanları sırayla gezmek istediğinizde — indeksin kendisine ihtiyaç duymadan — çok daha sade bir biçim kullanabilirsiniz:

```
22  int sayilar[] = {10, 20, 30, 40};
23  for (int sayi : sayilar) {
24      std::cout << sayi << '\n';
25  }
```

Aralık tabanlı döngü: Bir dizi/koleksiyonun elemanlarını indeks yazmadan tek tek gezen `for` biçimi (*range-based for*); sözdizimi `for (Tip eleman : koleksiyon) { ... }` biçimindedir.

Aralık tabanlı döngü bu satırda `sayilar` dizisinin dört elemanını sırayla, hiçbir indeks değişkeni tanımlamadan gezer: her turda `sayi` değişkeni dizinin bir sonraki elemanının bir *kopyasını* tutar, döngü gövdesi bu kopyayı ekrana basar. Klasik döngüyle kıyaslayın; şöyle yazmanız gerekirdi:

```
1  for (int i = 0; i < 4; i++) { std::cout << sayilar[i] << '\n'; }
```

Burada hem `i`'nin başlangıç/bitiş/artış değerlerini elle doğru yazmanız hem de `sayilar[i]` ile indeksleme yapmanız gerekiyordu. Aralık tabanlı döngüde bunların hiçbiri yoktur — indeks hatası yapma ihtimali baştan ortadan kalkar, çünkü indeksin kendisi hiç yazılmaz.

Bir de süslü parantezli ilklenmeye (`{}`) yüzeysel bir göz atın: yukarıdaki örnekte `int sayilar[] = {10, 20, 30, 40};` satırında dizinin elemanlarını `{}` içinde art arda yazdınız — bu sözdizimi C'den de tanıdıktır, ama C++'ta `{}` ile ilklemeyi yalnızca dizilerde değil, ileride göreceğiniz kullanıcı tanımlı tiplerde de (Hafta 3'ün `Nokta a{1.0, 2.0};` gibi) tutarlı biçimde kullanacaksınız — bu haftaki Ünite 4'te bu kalıbı bizzat kendi `Nokta` tipinizde göreceksiniz.

Bu ünitede gördüğünüz fikir kısaca şudur: `auto` tip adını elle yazma işini derleyiciye devreder, ama tipin kendisini değil, yalnızca kim yazdığını değiştirir; `const` bir değeri değiştirmeyeceğinize dair verdiğiniz sözü derleyiciye zorla uygulatır, ve bir referansla birleştiğinde (`const T&`) hem kopyalama maliyetinden kaçınma hem de değişmezlik garantisini bir arada verir; aralık tabanlı `for` ise indeks yazmadan bir koleksiyonu gezmenizi sağlar. Gelecek ünitede bu araçların hepsini bir arada kullanarak, bu haftanın `Nokta` örneğini gerçek bir kütüphane parçasına — `.hpp`/`.cpp` ve `namespace` ile — dönüştüreceksiniz.

3 • Katılım **Şimdi Siz Deneyin**

(a) `auto a = 7; auto b = 2.5; auto toplam = a + b;` yazdığınızda `toplam`'ın tipi nedir, neden? (b) `const std::string&` alan bir fonksiyon, değere göre (`std::string`) alan bir fonksiyona kıyasla hangi iki faydayı birden sağlar?

Geribildirim. (a) `toplam` bir `double`'dır: `a` `int` (`7`'den), `b` `double` (`2.5`'ten) çıkarılır; `a + b`'de `int`, olağan aritmetik dönüşümle `double`'a yükseltilir, sonuç `double` olur (`9.5`) ve `auto` bu sonuçtan `toplam`'ın tipini `double` olarak belirler. `auto`, tipi derleme zamanında tek ve sabit olarak saptar — dinamik değildir. (b) İki fayda: kopyalama maliyetinden kaçınma (referans olduğundan kopya yapılmaz) ve fonksiyonun parametreyi değiştirmeyeceği garantisi (`const` olduğundan yazma girişimi derleme hatasıdır).

Kendinizi Sınayın 1.3

`const std::string&` parametreli bir fonksiyon, değere göre (`std::string`) alan bir fonksiyona kıyasla hangi iki faydayı birden sağlar?

5 • İzleme **Aktarım — Başka Nerede?**

`const` doğruluğu dönem boyunca sürecek bir alışkanlıktır: Kitaplık projesinde bir `Kitap`'ı yazdıran ya da inceleyen her fonksiyon parametreyi `const Kitap&` olarak alacak — kopyasız ve değişmez. `auto` ise özellikle ileride kaplarla (`std::vector` gibi) ve yineleyici tipleriyle çalışırken uzun tip adlarını gizlemek için tekrar tekrar karşınıza çıkacak; bu ünitenin aralık tabanlı `for` tadımlığı, o kullanımın ilk işaretidir.

1.4 Kütüphaneyi Kurmak: Header/Source, `namespace`, CMake

1 • Ön-öğretim **Neden Önemli?**

Her şeyi tek bir `main.cpp`'ye yazarsanız dosya büyür, kod başka projelerde yeniden kullanılamaz ve birden çok kişi aynı anda çalışamaz. Bildirimi (arayüz) tanımdan (uygulama) ayırıp kodu birden çok `.cpp`'ye bölmek gerekir — ama derlerken elle `g++ a.cpp b.cpp c.cpp ...` yazmak, dosya sayısı arttıkça sürdürülemez hâle gelir. Bu ünite `.hpp`/`.cpp` ayrımı ve CMake ile bu düzeni gerçek, derlenebilir bir proje olarak kurar.

1 • Ön-öğretim **Ön Bilgi**

Bu ünite önceki üçünü tek bir yerde birleştirir; şunları taze tutun: Ünite 1'in derleme + bağlama modeli ve başlığın yalnız bildirim taşıma rolü (burada somutlaşır), Ünite 2'nin `namespace`'i (`Nokta`'yı `geometri` ad alanına koymak için), Ünite 3'ün `const` referansı (`topla`/`yazdir` imzalarında kullanılır). C'den ise, varsa, birden çok dosyayı ayrı ayrı derleyip bağlama deneyiminizi anımsayın.

Bu haftaki üç ünitede tek tek gördüğünüz araçları — derleme + bağlama modeli, `namespace`, referanslar, `auto`, `const` — şimdi bir arada, gerçek, derlenebilir bir proje içinde kullanacaksınız. Bu ünitenin örneği, C'deki bir `struct Nokta`'ya çok benzeyen, iki boyutlu bir nokta değeridir: `geometri` ad alanına yerleştirilmiş, henüz bir *sınıf* olmayan (Hafta 3'e kadar sınıf görmeyeceksiniz), yalnız birkaç `double` alan taşıyan basit bir `struct`. Amacınız bu yapıyı ve onun üzerinde çalışan birkaç serbest fonksiyonu, tek bir dosyada değil, bir `.hpp`/`.cpp` çifti ve bir `CMakeLists.txt` ile gerçek bir kütüphane parçası hâline getirmek.

1.4.1 `.hpp`/`.cpp`: bildirim ile tanımın ayrılması

Ünite 1'de gördüğünüz derleme + bağlama modelini hatırlayın: bir `.cpp` dosyası başka bir dosyadaki bir tanımı kullanmak istediğinde, o tanımın *bildirimine* ihtiyaç duyar, gövdesine değil — gövdeyi bulup bağlamak bağlayıcının işidir. `Nokta` yapınız için bu ayrımı somut olarak kurun. Önce bildirimlerin durduğu **başlık dosyası**:

başlık dosyası: bir modülün dışa açtığı bildirimleri (arayüzü)[7] barındıran dosya[8] (*header file*); geleneksel[9] uzantısı `.hpp`, gövde[10] içermez.

```
struct Nokta {
    double x;
    double y;
};
```

Bu, C'deki bir `struct` tanımından hiç farklı görünmüyor — ve gerçekten de öyle: `Nokta`, iki `double` alan (`x`, `y`) taşıyan, henüz hiçbir üye fonksiyonu, hiçbir `private`/`public` bölümü olmayan bir *aggregate struct*'tır. Bu bilinçli bir tercih — bu hafta sınıf kavramını henüz görmediniz (Hafta 3'te `class` anahtar kelimesiyle tanışacaksınız); `Nokta` şimdilik yalnızca bir veri paketidir, tıpkı C'deki bir `struct` gibi. Aynı dosyada, `Nokta` üzerinde çalışacak iki serbest fonksiyonun bildirimleri de durur:

```
14 Nokta topla(const Nokta& a, const Nokta& b);
15 void yazdir(const Nokta& n);
```

Dikkat ederseniz bu iki satırda yalnızca fonksiyonların *imzaları* var — adları, parametre tipleri, dönüş tipleri — gövdeleri yok. `topla`, iki `const Nokta&` parametre alıp yeni bir `Nokta` döndürür (parametrelerin neden `const` referans olduğunu bir sonraki bölümde göreceksiniz); `yazdir` bir `const Nokta&` alıp hiçbir şey döndürmez (`void`), yalnızca bir yan etki — ekrana yazma — yapar. `nokta.hpp`'yi include eden herhangi bir dosya, bu iki fonksiyonun *var olduğunu* ve nasıl çağrılacağını bilir, gövdelerinin gerçekte nasıl yazıldığını bilmesine hiç gerek yoktur.

Gövdelerin gerçek yeri, kaynak dosyasıdır:

```
8  Nokta topla(const Nokta& a, const Nokta& b) {
9      return Nokta{a.x + b.x, a.y + b.y};
10 }
```

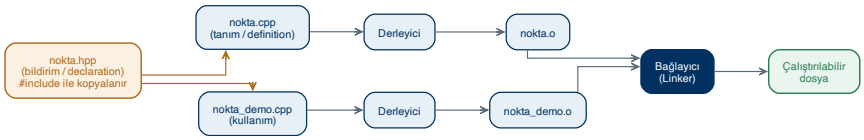
```
14 void yazdir(const Nokta& n) {
15     std::cout << "(" << n.x << ", " << n.y << ")";
16 }
```

`topla`'nın gövdesini adım adım okuyun: iki `Nokta`'nın `x` ve `y` bileşenlerini ayrı ayrı toplayıp yeni bir `Nokta`'yı süslü parantezli ilklenmeyle (`Nokta{a.x + b.x, a.y + b.y}`) oluşturup döndürür — Ünite 3'te gördüğünüz `{}` sözdiziminin bir kullanıcı tanımlı tipte (burada: `struct Nokta`) uygulanmış hâlidir. `yazdir`'ın gövdesi ise iki alanı `(x, y)` biçiminde ekrana basar; her iki fonksiyon da parametrelerini yalnızca *okur*, hiçbirini değiştirmez — bu yüzden ikisi de `const Nokta&` alır: kopyasız (referans olduğu için) ve güvenli (`const` olduğu için, Ünite 3'te gördüğünüz `const T&` kalıbının tam burada bir uygulaması).

Analoji

Header/source ayrımını bir restoran menüsü ile mutfağa benzetebilirsiniz. Menü (`.hpp`) müşteriye nelerin sipariş edilebileceğini — yemeğin adını, ne içerdiğini — gösterir; gerçek hazırlanış (`.cpp`, yani mutfak) müşteriden gizlidir. Müşteri mutfağa hiç girmeden, yalnızca menüye bakarak sipariş verebilir — `nokta_demo.cpp` dosyanız da tam olarak böyle davranacak: `nokta.hpp`'yi include edip `topla`/`yazdir`'ı çağıracak, ama gövdelerinin nasıl yazıldığını hiç bilmeyecek.

Şekil 1.3 bu ayrımı Ünite 1'in derleme + bağlama şemasıyla aynı görsel dilde, ama somut `nokta.hpp`/`nokta.cpp` örneğiyle özetliyor: tek bir `.hpp` bildiriminden iki ayrı `.cpp` dosyasına (kullanan ve tanımlayan) giden include okları, her biri ayrı ayrı derlenip kendi `.o` dosyasını üretiyor, bağlayıcı sonunda ikisini tek çalıştırılabilirde birleştiriyor.



Şekil 1.3: Header/source ayrımı: bir `.hpp` bildiriminden iki `.cpp` dosyasına include, ayrı derleme, bağlamada birleşme.

1.4.2 `#pragma once`: çoklu include koruması

`nokta.hpp` dosyasının en tepesindeki `#pragma once` satırını soyut bırakmayın; çözdüğü sorunu küçük bir senaryoyla somutlaştırın. Önişlemcinin `#include` karşısındaki işi son derece basittir: `#include "nokta.hpp"` satırının yerine, `nokta.hpp`'nin *tüm içeriğini* olduğu gibi yapıştırır — bir metin editöründe kopyala-yapıştır yapmış gibi. Şimdi düşünün ki `nokta_demo.cpp` dosyanız, başka başka nedenlerle, `nokta.hpp`'yi *iki farklı yerden* — doğrudan kendisi *ve* `nokta.hpp`'yi zaten include eden başka bir başlık üzerinden dolaylı olarak — include etmek zorunda kalsaydı: bu durumda önişlemci, `struct Nokta` tanımını *aynı çeviri birimine iki kez* yapıştırırdı. Derleyici bu ikinci kopyayla karşılaştığında "yeniden tanımlama" (*redefinition*) hatası verir — `struct Nokta` zaten bir kez tanımlanmışken, aynı isimle ikinci bir tanım daha görür ve buna itiraz eder.

Çoklu include koruması: aynı header'ın bir çeviri biriminde birden fazla kez işlenmesini engelleyen mekanizma (*include guard*); en yaygın modern biçimi `#pragma once` satırıdır.

Çoklu include koruması tam olarak bu sorunu çözer. `#pragma once`, derleyiciye "bu dosyayı bir çeviri biriminde yalnızca bir kez oku" der: önişlemci `nokta.hpp`'yi bir kez yapıştırdıktan sonra, aynı çeviri birimi içinde bu dosyaya gelen her sonraki `#include` satırını sessizce atlar — `struct Nokta` tanımı hiçbir zaman ikinci kez yapıştırılmaz, "yeniden tanımlama" hatası hiç oluşmaz.

Analoji

`#pragma once`'u bir etkinlikte tek seferlik giriş damgasına benzetebilirsiniz. Aynı davetliye elde ikinci kez damga vurulmaz — görevli "bu kişi zaten damgalandı" deyip ikinci damgalama girişimini sessizce atlar. `#pragma once` da header'ın ikinci kez include edilmeye çalışıldığı anı tam olarak böyle karşılar: "bu dosya zaten işlendi" deyip ikinci kopyalamayı atlar.

Satırın # ile başlaması tesadüf değildir: bu bir C++ ifadesi değil, kaynak kod derleyiciye gitmeden önce çalışan önişlemciye verilmiş bir talimattır — tıpkı Ünite 1'de gördüğünüz #include gibi.

Ortak Hata / Tuzak

#pragma once satırını unutup header'ı "çıplak" bırakmak, küçük tek dosyalık projelerde uzun süre fark edilmeyebilir — çünkü hata yalnızca o header birden fazla yerden (doğrudan ya da dolaylı olarak) include edildiğinde ortaya çıkar. Proje büyüdükçe, bir başlığın başka bir başlık tarafından da include edilmesi kaçınılmaz hâle gelir; bu noktada #pragma once eksikse "yeniden tanımlama" hatası genellikle beklenmedik, izi sürülmesi zor bir anda ortaya çıkar. Kural basit: yazdığınız her .hpp dosyasının ilk satırı, istisnasız, #pragma once olsun.

1.4.3 namespace geometri: isim düzenleme

nokta.hpp ve nokta.cpp dosyalarının ikisinde de, struct Nokta tanımının ve serbest fonksiyonların çıplak durmadığını, bir namespace geometri { ... } bloğunun içine yerleştirildiğini fark etmişsinizdir. Ünite 2'de gördüğünüz ad alanı fikrini burada somut bir senaryoyla pekiştirin: Nokta kısa ve genel bir isimdir — dersin başka bir alıştırmasında ya da paylaştığınız bir kod parçasında, tamamen farklı bir amaçla (örneğin bir oyun ya da bir grafik kütüphanesinde) yine Nokta adında ayrı bir tip tanımlanmış olabilir. Her iki Nokta da ad alanı olmadan, doğrudan global kapsamda tanımlanmış olsaydı, bu iki başlığı aynı .cpp dosyasında include etmeniz gerektiğinde derleyici aynı adı taşıyan iki farklı tanımla karşılaşırdı. geometri ad alanına yerleştirmek bu çakışmayı baştan imkânsız kılar: Nokta adını geometri ad alanına sabitlersiniz; dışarıdan erişmek istediğinizde geometri::Nokta yazarsınız — tıpkı nokta_demo.cpp'de göreceğiniz gibi. Öteki Nokta kendi ad alanına (örneğin oyun::Nokta) yerleştirilmiş olsaydı, iki tip aynı kısa adı taşısa bile derleyici bunları asla karıştırmaz, çünkü tam nitelenmiş adları — geometri::Nokta ile oyun::Nokta — birbirinden tamamen farklıdır.

Bu düzenin dönem boyunca tekrar tekrar karşınıza çıkacağını şimdiden bilin: Hafta 3'te ilk gerçek sınıfınızı — Kitap'ı — tanıttığınızda, aynı kalıbı kitaplik ad alanıyla bire bir tekrarlayacaksınız — kendi .hpp/.cpp çiftinde yaşayan, en tepesinde #pragma once bulunan, bir namespace bloğunun içine yerleştirilmiş bir tanım. Bu haftaki geometri::Nokta örneği, o kalıbın — henüz sınıf içermeyen, ama tüm iskeleti önceden kuran — ilk denemesidir.

1.4.4 CMake ile ilk derlenebilir proje

Şimdi nokta.hpp, nokta.cpp ve bunları kullanan bir main fonksiyonunu — nokta_demo.cpp — tek bir çalıştırılabilir dosyada bir araya getirmeniz gerekiyor. Ünite 1'de kavramsal düzeyde tanıdığınız CMake'i şimdi gerçek bir dosya üzerinde görün. Bu haftaki CMakeLists.txt dosyası beş satırdan oluşur: cmake_minimum_required(VERSION 3.20) gereken en düşük CMake sürümünü bildirir; project(hafta01_temel CXX) projenin adını ve bir C++ projesi olduğunu belirtir; set(CMAKE_CXX_STANDARD 20) ve set(CMAKE_CXX_STANDARD_REQUIRED ON) derleyiciye C++20 standardını, ödünsüz biçimde, kullanmasını söyler; son olarak add_executable(nokta_demo nokta_demo.cpp nokta.cpp) satırı nokta_demo adlı çalıştırılabilir dosyanın hangi kaynak dosyalarından derleneceğini belirtir.

Bu son satırı dikkatle okuyun: kaynak listesinde `nokta_demo.cpp` *ve* `nokta.cpp` vardır, ama `nokta.hpp` *yoktur*. Bunun nedenini Ünite 1'in derleme modeliyle bağlayın: bir başlık dosyası kendi başına derlenmez, yalnızca onu include eden `.cpp` dosyalarının içine önişlemci tarafından kopyalanır; CMake'e "derle" demeniz gereken şey yalnızca gerçek `.cpp` kaynaklarıdır. `nokta.cpp`'yi bu listeden çıkarsaydınız ne olurdu, adım adım düşünün: `nokta_demo.cpp` yine derlenirdi — çünkü derleyici yalnızca `nokta.hpp`'deki bildirimlere bakar, `topla`/`yazdir`'ın gövdelerine ihtiyaç duymaz — ama bağlama aşamasında bağlayıcı bu iki fonksiyonun gövdesini hiçbir nesne dosyasında bulamayacağı için "tanımsız sembol" (*undefined reference*) hatasıyla durur. Kısacası: derleme aşaması yalnızca bildirimlerle yetinir, bağlama aşaması ise gövdelerin gerçekten bir yerde var olmasını zorunlu kılar.

Şimdi bu üç dosyanın nasıl bir araya geldiğini, kullanım noktasında görün:

```
7   geometri::Nokta a{1.0, 2.0};
8   geometri::Nokta b{3.0, 4.0};
9   auto toplam = geometri::topla(a, b);
10
11  geometri::yazdir(a);
12  std::cout << " + ";
13  geometri::yazdir(b);
14  std::cout << " = ";
15  geometri::yazdir(toplam);
16  std::cout << '\n';
```

Bu satırları sırayla izleyin. İlk iki satır iki `geometri::Nokta` nesnesi oluşturur — `a{1.0, 2.0}` ve `b{3.0, 4.0}` — Ünite 3'te gördüğünüz süslü parantezli ilklenmeyle. Üçüncü satırda `auto toplam = geometri::topla(a, b);` yazdığınızda, `auto` `topla`'nın dönüş tipini — `geometri::Nokta` — sizin yerinize çıkarır; tip adını elle yazmanız gerekmez. Sonraki satırlar `geometri::yazdir`'ı üç kez çağırıp `a`, `b` ve `toplam`'ı ekrana basar. Her çağrıda `geometri::` önekini görüyorsunuz — `using namespace geometri;` yazmadığınız için, Ünite 2'de gördüğünüz güvenli, açık yaklaşımı burada da sürdürüyorsunuz. Bu programı derleyip çalıştırdığınızda ekranda şuna benzer bir satır görürsünüz:

Beklenen Çıktı

```
(1, 2) + (3, 4) = (4, 6)
```

Dene-Çalıştır

Bu haftaki `kod_ornekleri/hafta01/` klasöründe CMake ile derleyin: önce `cmake -S . -B build` ile bir derleme klasörü oluşturun, ardından `cmake --build build` ile derleyin; son olarak üretilen `nokta_demo` çalıştırılabilir dosyasını (`build/` klasörü içinde) çalıştırın. Yukarıdaki çıktı satırını görmelisiniz. İsterseniz aynı derlemeyi CMake'siz, doğrudan derleyiciyi çağırarak da deneyin: `clang++ -std=c++20 nokta_demo.cpp nokta.cpp -o nokta_demo` — iki yaklaşım da aynı sonucu üretir; CMake yalnızca bu iki adımı (derleme ve bağlama) sizin için otomatik sıralar.

İpucu

Bu hafta kurduğunuz iskelet — `.hpp`/`.cpp` ayrımı, `#pragma once`, bir `namespace` bloğu, `add_executable`'lı bir `CMakeLists.txt` — bir kerelik bir alıştırma değil-

dir. Hafta 3'te `Kitap` sınıfını tam olarak bu kalıpla, `kitaplik` ad alanı içinde kuracaksınız; dönem boyunca kütüphanenize eklediğiniz her yeni parça aynı desende — kendi `.hpp`/`.cpp` çiftinde, `namespace` içinde, `CMakeLists.txt`'nin kaynak listesine eklenmiş bir satırla — yaşayacak. `geometri::Nokta`, henüz bir sınıf değil, ama bu deseni öğrenmenin ilk, en basit örneğiydi.

Bu ünitede gördüğünüz fikir kısaca şudur: `.hpp`/`.cpp` ayrımı Ünite 1'in derleme + bağlama modelini somut bir dosya düzenine dönüştürür — bildirim her çeviri birimine kopyalanır, tanım yalnızca bir kez derlenir; `#pragma once` bu ayrımı çoklu include'a karşı güvenceye alır; `namespace geometri` ise `Nokta` gibi genel bir adı isim çakışmasından korur; `CMakeLists.txt` ise bu dosyaları tek bir okunaklı komutla derlenebilir tutar. Bu dört ünitede gördüğünüz her araç — derleme modeli, `namespace`/akışlar/referanslar, `auto`/`const`, ve şimdi bu kütüphane iskeleti — gelecek haftadan itibaren C++'ın bellek modelini, fonksiyon aşırı yüklemesini ve sonunda (Hafta 3'te) gerçek sınıfları işlerken sizin zeminiz olacak.

3 · Katılım **Şimdi Siz Deneyin**

`nokta.hpp`'yi hem `nokta.cpp` hem `nokta_demo.cpp` include ediyor. (a) `#pragma once` koymazsanız ve başlık dolaylı olarak iki kez dahil edilirse ne olur? (b) `CMakeLists.txt`'te `nokta_demo` hedefini oluştururken neden hem `nokta_demo.cpp` hem `nokta.cpp` dosyasını eklemek zorundasınız?

Geribildirim. (a) Aynı `struct`/fonksiyon bildirimleri o çeviri biriminde iki kez ortaya çıkar; bu "yeniden tanımlama" (*redefinition*) bir derleme hatasıdır. `#pragma once`, başlığın çeviri birimi başına yalnız bir kez işlenmesini sağlayarak bunu önler. (b) `topla`/`yazdir` gövdeleri `nokta.cpp`'de derlenir; hedefe yalnız `nokta_demo.cpp` verirseniz bu gövdeler hiç derlenmez, bağlayıcı onları bulamaz ve "undefined reference" verir — bu, Ünite 1'in bağlama aşaması hatasının aynısıdır. İki `.cpp` de hedefe eklenmelidir.

Kendinizi Sınayın 1.4

`#pragma once` olmasaydı, aynı header'ı — doğrudan ve dolaylı olarak — iki farklı yerden include eden bir `.cpp` dosyasında ne olurdu?

5 · İzleme **Aktarım — Başka Nerede?**

Bu hafta kurduğunuz `.hpp`/`.cpp` + `namespace` + CMake iskeleti, dönem boyunca büyüyecek kişisel kütüphanenizin temel desenidir. Hafta 3'te `Kitap` sınıfını tam olarak aynı kalıpla — `kitaplik` ad alanında, `kitap.hpp`/`kitap.cpp` olarak — oluşturacak; ileride her yeni bileşende (`Vektor2B`, `Materyal` …) aynı deseni yeniden kullanacaksınız. Bugün kurduğunuz iskelet, Kitaplık projesinin ilk taş sırasıdır.

Bölüm Özeti

Özet

Bu hafta C'den C++'a köprüyü kurdunuz. İlk ünitede gördünüz ki C++, çalışma modeli olarak C'den ayrılmaz — ikisi de derlenip bağlanır — ama derleyicinin size verdiği sözler çok daha sıkıdır: tip güvenliği, ve mekanizmasını ileride göreceğiniz RAII fikri, bu sıkılığın ilk işaretleridir; derleme + bağlama modelini ve header dosyalarının yalnızca bildirim taşıyıp önişlemci aşamasında kopyalandığını burada öğrendiniz. İkinci ünitede ilk gerçek C++ araçlarınızı elinize aldınız: `namespace` isim çakışmasını çözer, `<iostream>` akışları (`std::cin`/`std::cout`) C'nin format dizgesine dayalı `scanf`/`printf`'ine kıyasla tip güvenli bir giriş/çıktı sağlar, `std::string` C'nin ham karakter dizisinden çok daha rahat bir metin tipidir, referanslar ise boş olamayan, yeniden bağlanamayan bir takma ad olarak C işaretçilerinden ayrılır. Üçüncü ünitede `auto` ile derleyicinin tip çıkarımına ve `const` ile değişmezlik sözüne baktınız — `const T&` kalıbının hem kopyalama maliyetinden kaçınma hem değiştirmeme garantisini bir arada verdiğini gördünüz; aralık tabanlı `for` ile indekssiz gezinmeyi de tanıdınız. Dördüncü ünite bu araçların hepsini bir araya getirdi: `geometri::Nokta` örneğini `.hpp`/`.cpp` olarak ayırdınız, `#pragma once` ile çoklu include'a karşı korudunuz, `namespace geometri` içine yerleştirdiniz ve `CMakeLists.txt` ile derlenebilir bir proje kurdunuz — bu, dönem boyunca büyüyecek kütüphanenizin, henüz sınıf içermeyen, ama tüm iskeleti önceden kuran ilk denemesiydi. Gelecek hafta fonksiyonları derinleştirip (aşırı yükleme, referans/pointer parametreler) C++'ın bellek modelini — stack/heap, `new`/`delete` ve RAII'nin ilk somut örneğini — işleyeceksiniz; bu haftanın referans ve RAII değinileri o haftanın doğrudan zemini, `.hpp`/`.cpp` + CMake iskeleti ise Hafta 2 ve sonrasında aynen tekrar kullanacağınız kalıptır.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — derleme neden iki aşamadır). C++'ta derleme tek adım değil de derleme + bağlama olarak iki ayrı aşamadır, çünkü her `.cpp` dosyası — kendi çeviri birimi olarak — diğerlerinden *habersiz*, kendi başına derlenir ve kendi nesne dosyasını (`.o`) üretir; derleyici bir dosyayı derlerken başka bir dosyada ne olduğunu bilmek zorunda değildir, yalnızca o dosyanın kullandığı isimlerin *bir yerde* tanımlı olduğuna dair bildirimlere (header'lara) güvenir. Bağlama aşaması ise bu ayrı ayrı üretilmiş nesne dosyalarını — ve varsa kütüphaneleri — birleştirip tek bir çalıştırılabilir dosya oluşturur; bir dosyadaki çağrının gerçekte hangi nesne dosyasındaki gövdeye gideceğini bu aşamada bağlayıcı çözer. Header dosyaları bu modelde yalnızca *bildirim* taşır — gövde içermezler — ve önişlemci tarafından, `#include` satırının bulunduğu yere, ilgili çeviri birimine olduğu gibi kopyalanır; bu sayede bir dosya başka bir dosyada tanımlı bir fonksiyonu, o fonksiyonun gövdesini hiç görmeden, yalnızca imzasına bakarak güvenle çağırabilir.

Soru (Ünite 2 — `cin >>` ile `scanf` arasındaki fark). `std::cin >> sayi;` ile C'nin `scanf("%d", &sayi);`'ı arasındaki temel fark tip güvenliğidir. `scanf` bir *format dizgesine* dayanır — siz `%d`, `%s` gibi bir belirteci elle seçer, değişkenin adresini (`&sayi`) elle verirsiniz; eğer format belirteci değişkenin gerçek tipiyle uyuşmazsa (`%d` yerine `%s` yazmak gibi) derleyici buna *hiç* itiraz etmez, hata yalnızca çalışma zamanında —

genellikle programın çökmesi ya da anlamsız bir değer okuması biçiminde — ortaya çıkar. `std::cin >> sayi;` yazarken ise ne bir format belirteci ne bir `&` yazarsınız; `>>` operatörü `sayi`'nin derleme anında bilinen gerçek tipine bakarak doğru okuma biçimini kendisi seçer. Böylece format-tip uyumsuzluğu riski — C'de çalışma zamanına kadar gizli kalabilen bir hata sınıfı — C++ akışlarında kökten ortadan kalkar.

Soru (Ünite 3 — const std::string& iki faydası). `const std::string&` parametreli bir fonksiyon, değere göre (`std::string`) alan bir fonksiyona kıyasla iki faydayı birden sağlar. Birincisi, kopyalama maliyetinden kaçınma: referans olduğu için fonksiyon çağrıldığında dizgenin tam bir kopyası oluşturulmaz, yalnızca çağırandaki gerçek dizgeye ikinci bir isim verilir — dizge uzunsa bu, her çağrıda tekrarlanan gereksiz bir bellek/zaman maliyetinden kurtulmak demektir. İkincisi, değiştirmeme garantisi: `const` sayesinde fonksiyon gövdesi bu referans üzerinden çağırandaki veriyi *değiştiremez* — derleyici bunu zorla uygular, gövdede bir yazma girişimi (`mesaj += "!";` gibi) derlenmez. Bu iki fayda birbirinden bağımsız değil, aynı anda elde edilir: referans olması kopyayı önler, `const` olması ise değişmezliği garanti eder — yalnızca okuyacağınız, tipi küçük olmayan her parametrede bu kalıp neredeyse her zaman doğru tercihtir.

Soru (Ünite 4 — `#pragma once` olmasaydı). `#pragma once` olmasaydı, aynı header'ı — doğrudan ve dolaylı olarak — iki farklı yerden include eden bir `.cpp` dosyasında önişlemci o header'ın içeriğini aynı çeviri birimine *iki kez* yapıştırırdı: bir kez doğrudan include satırından, bir kez de o header'ı zaten include eden başka bir başlık üzerinden dolaylı olarak. Sonuç, aynı `struct`/ fonksiyon bildirimlerinin aynı dosyada iki kez görünmesidir ve derleyici buna "yeniden tanımlama" (*redefinition*) hatasıyla itiraz eder — program hiç derlenmez. `#pragma once`, derleyiciye "bu dosyayı bir çeviri biriminde yalnızca bir kez oku" diyerek bu ikinci yapıştırmayı baştan engeller; önişlemci dosyayı bir kez işledikten sonra, aynı çeviri birimi içindeki her sonraki `#include` satırını sessizce atlar.